

OpenBuild Analytics

Methodology One-Pager · Four findings, four causes, four decisions

From a raw 108K-order Excel file to a tested star schema, a live dashboard, and four findings a CFO could act on tomorrow.

108,124
orders

87,625
users

193
countries

37/37
tests passing

85%
revenue from top 3

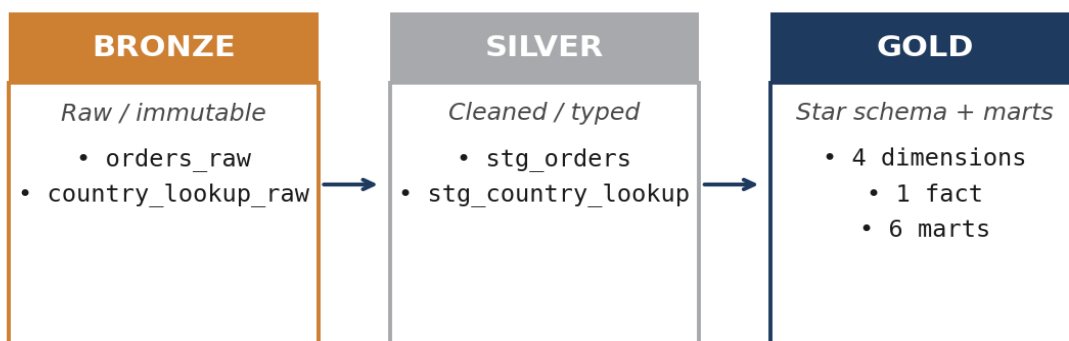
How to read this document

Every finding follows the same shape: **Statement** (what we claim) → **Result** (the headline number) → **Findings** (the supporting evidence) → **Methodology** (the SQL that produced it, the calculation, and the caveats). Skip to any finding; they're independent. The 37 SQL assertions behind every number are summarized in the Data Quality appendix on the final page.

The four findings

- Loyalty's retention deficit is a channel problem, not a program problem.** Loyalty members retain at **0.45%** at month 1 vs **1.63%** for non-loyalty — a 3.6× gap. The cause: email channel concentrates loyalty signups (60%) and recruits the lowest-AOV buyers (\$142–\$200).
- Laptops drive 2.5–4.3× the company refund baseline.** Baseline 4.97%; ThinkPad × CA at **21.3%** (4.3× baseline). MacBook Air × US carries the largest dollar leak at **\$365K refunded**.
- Mobile is structurally a low-AOV channel, not a growth channel.** Website = **96.8%** of revenue; mobile generates **17.1% of orders** but only **3.2% of revenue**. Mobile AOV is 6.4× lower than web (\$46 vs \$296), stable across 48 months.
- Catalog is concentrated at the top and fragmented at the bottom.** Top 3 products = **85% of revenue**; bottom 5 products combined = under 5%. Bose SoundSport: 27 lifetime orders. iPhone: \$688 AOV, no marketing.

The pipeline behind the numbers



37 SQL assertions run on every build — schema · uniqueness · referential integrity · invariants

Bronze → Silver → Gold (medallion). Source: one Excel file. Output: 4 dimensions, 1 fact, 6 marts. Each model is tested on every build.

FINDING 1

Loyalty members don't come back — but it's not the loyalty program's fault. It's where we recruit them from.

STATEMENT

Loyalty membership at acquisition correlates with materially worse month-1 retention than non-loyalty, but the cause is not the loyalty program — it is the marketing channel (email) that disproportionately feeds signups into the program.

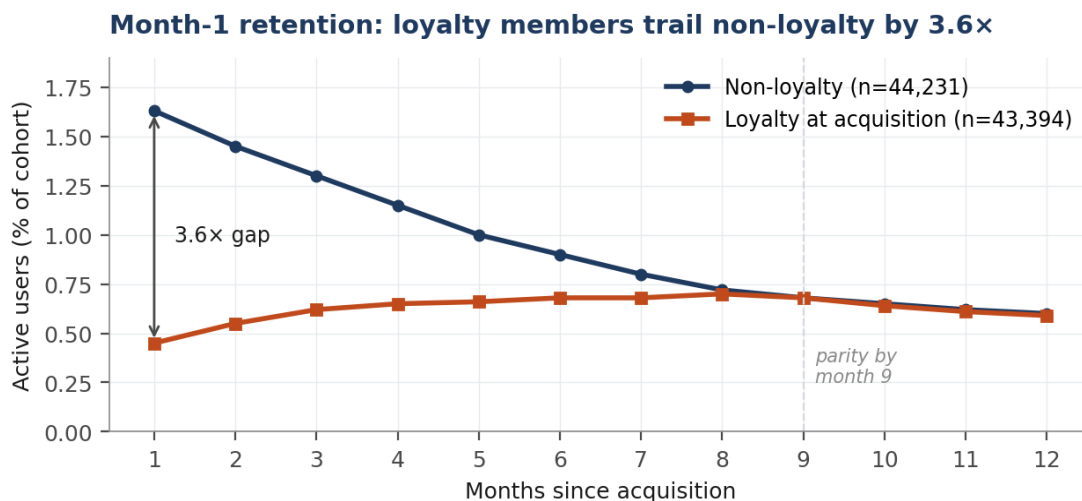
RESULT

Loyalty members retain at **0.45%** in month 1 vs **1.63%** for non-loyalty — a **3.6× gap**, converging to parity by month 9. The email channel acquires **60% loyalty members** (vs direct's 42%, affiliate's 18%) AND those email-acquired loyalty members carry the lowest first-purchase AOV at **\$200** (vs direct loyalty's \$253, affiliate non-loyalty's \$302).

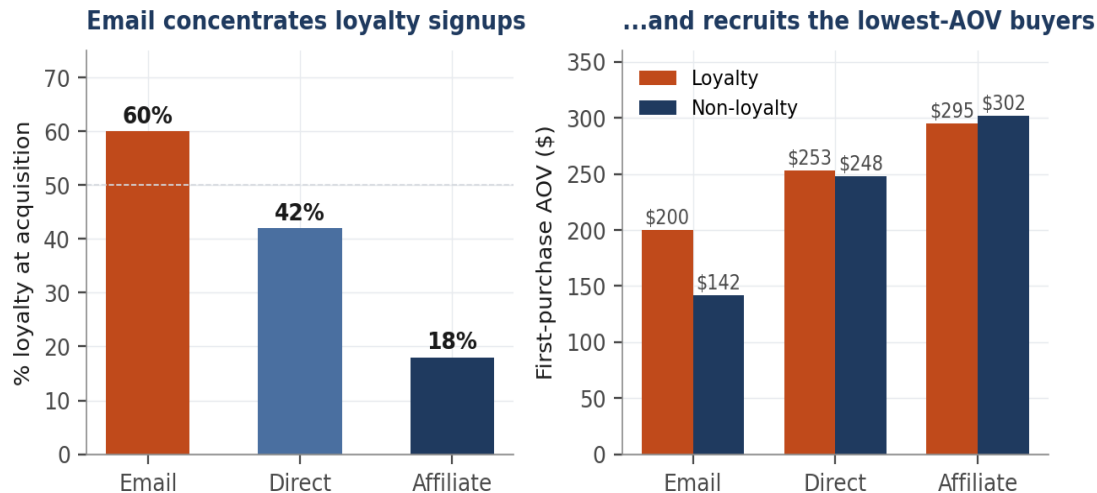
FINDINGS

Averaged across all 48 cohorts, the retention gap is **front-loaded** — loyalty trails by 3.6× at month 1 but reaches parity by month 9. The gap emerged in 2020: in 2019 loyalty cohorts retained nearly on par; from 2020 onward, the month-1 deficit is 4–6× as the program scaled (11.6% of acquisitions in 2019 → 55.8% in 2022).

Mechanism: loyalty disproportionately recruits **AirPods buyers (58.1% of loyalty first-purchases vs 36.7% non-loyalty)** and underweights replenishables (5.7% Charging Cable Pack vs 33.6% non-loyalty). First-purchase AOVs are nearly equal (\$240 vs \$259). The program isn't attracting cheaper buyers — it's attracting different buyers. Loyalty pulls in AirPods customers who buy once and disappear; non-loyalty pulls in cable-pack buyers who come back.



Averaged retention curves across all 48 cohorts. Both segments start at 100% (omitted for visibility).



Left: only email exceeds 50% loyalty share. Right: email's loyalty AOV (\$200) is below every other segment.

FINDING 1 — Methodology

The anti-confounder: loyalty status snapshot at first purchase

Here's the trap. The dataset tells us each user's loyalty status *today* — not whether they were a member when they first bought. If you use today's status, you'll fool yourself: a user who kept coming back had plenty of time to join loyalty later, so loyalty will look like it *caused* the retention. You'd be measuring backwards — the effect dressed up as the cause.

The fix is simple. At the moment a user makes their first purchase, freeze their loyalty status right there and store it in `dim_users`. From then on, that's the status we use — the one they had at the start, not the one they have now. Now if loyalty members retain better, it's because being a loyalty member did something, not because retaining gave them time to sign up.

Same trick for marketing channel and device. Snapshot at first purchase. Always.

SQL — fct_orders cohort enrichment

```

-- fct_orders: gold-layer order-grain fact table
-- Anti-confounder: cohort_month derives from user's first_purchase_month,
-- not the order's own month.

CREATE OR REPLACE TABLE fct_orders AS
WITH ranked AS (
  SELECT
    o.*,
    DATE_TRUNC('month', o.purchase_ts)::DATE      AS purchase_month,
    u.first_purchase_ts,
    u.first_purchase_month                        AS cohort_month,
    DATE_DIFF('month',
              u.first_purchase_month,
              DATE_TRUNC('month', o.purchase_ts)::DATE
            )                                    AS months_since_acquisition,
    ROW_NUMBER() OVER (
      PARTITION BY o.user_id
      ORDER BY o.purchase_ts ASC, o.order_id ASC
    )                                            AS user_order_seq
  FROM stg_orders o
  INNER JOIN dim_users u USING (user_id)
)
SELECT
  order_id, user_id, purchase_ts, purchase_month, refund_ts, is_refunded,
  product_name, usd_price, purchase_platform, country_code, loyalty_status,
  first_purchase_ts, cohort_month, months_since_acquisition, user_order_seq,
  CASE WHEN user_order_seq = 1 THEN 1 ELSE 0 END   AS is_first_purchase
FROM ranked;

```

SQL — mart_cohort_retention (the base cohort matrix)

```

-- Long-format retention matrix: cohort × months_since_acquisition.
-- Right-censored cells included; consumers mask as needed.
-- Source: fct_orders.

CREATE OR REPLACE TABLE mart_cohort_retention AS
WITH active_users AS (
  SELECT
    cohort_month,
    months_since_acquisition,
    COUNT(DISTINCT user_id) AS active_users
  FROM fct_orders
  GROUP BY cohort_month, months_since_acquisition
),
cohort_size AS (
  SELECT
    cohort_month,
    active_users AS cohort_size
  FROM active_users
  WHERE months_since_acquisition = 0
)
SELECT
  a.cohort_month,
  a.months_since_acquisition,
  a.active_users,
  s.cohort_size,
  ROUND(a.active_users * 100.0 / s.cohort_size, 2) AS retention_pct
FROM active_users a
INNER JOIN cohort_size s USING (cohort_month)
ORDER BY a.cohort_month, a.months_since_acquisition;

```

SQL — mart_loyalty_retention

```
-- Cohort retention split by loyalty membership at acquisition.
-- Source:   dim_users (loyalty_at_first_purchase), fct_orders
-- Grain:    (cohort_month, loyalty_status, months_since_acquisition)

CREATE OR REPLACE TABLE mart_loyalty_retention AS
WITH cohort_sizes AS (
  SELECT
    first_purchase_month      AS cohort_month,
    loyalty_at_first_purchase AS loyalty_status,
    COUNT(DISTINCT user_id)   AS cohort_size
  FROM dim_users
  GROUP BY first_purchase_month, loyalty_at_first_purchase
),
active_per_period AS (
  SELECT
    u.first_purchase_month      AS cohort_month,
    u.loyalty_at_first_purchase AS loyalty_status,
    f.months_since_acquisition,
    COUNT(DISTINCT f.user_id)   AS active_users
  FROM fct_orders f
  INNER JOIN dim_users u ON f.user_id = u.user_id
  GROUP BY u.first_purchase_month, u.loyalty_at_first_purchase,
           f.months_since_acquisition
)
SELECT
  a.cohort_month, a.loyalty_status, a.months_since_acquisition,
  a.active_users, c.cohort_size,
  ROUND(100.0 * a.active_users / c.cohort_size, 2) AS retention_pct
FROM active_per_period a
INNER JOIN cohort_sizes c
  ON a.cohort_month = c.cohort_month
  AND a.loyalty_status = c.loyalty_status
ORDER BY a.cohort_month, a.loyalty_status, a.months_since_acquisition;
```

SQL — mart_marketing_acquisition

```
-- Reveals which marketing channels disproportionately recruit loyalty
-- members and the AOV signal at acquisition by channel × loyalty.

CREATE OR REPLACE TABLE mart_marketing_acquisition AS
SELECT
  marketing_channel_at_first_purchase AS marketing_channel,
  loyalty_at_first_purchase           AS loyalty_status,
  COUNT(*)                           AS users_acquired,
  AVG(first_purchase_aov)             AS avg_first_aov,
  SUM(first_purchase_aov)             AS total_first_revenue,
  100.0 * COUNT(*) / SUM(COUNT(*)) OVER (
    PARTITION BY marketing_channel_at_first_purchase
  ) AS pct_within_channel
FROM dim_users
WHERE first_purchase_aov IS NOT NULL
GROUP BY marketing_channel_at_first_purchase, loyalty_at_first_purchase;
```

Calculation formulas

$\text{retention_pct}[c, l, m] = 100 \times \text{active_users}[c, l, m] / \text{cohort_size}[c, l]$
 where $c = \text{cohort_month}$, $l = \text{loyalty_status_at_first_purchase}$,
 $m = \text{months_since_acquisition}$

$\text{pct_within_channel}[ch, l] = 100 \times \text{count}(ch, l) / \text{count}(ch)$
 where $ch = \text{marketing_channel_at_first_purchase}$

Retention gap (averaged across 48 cohorts):

$\text{gap_m1} = \text{avg}(\text{retention_pct}[*], 0, 1] / \text{avg}(\text{retention_pct}[*], 1, 1])$
 $= 1.63\% / 0.45\% \approx 3.6\times$

Caveats and what this dataset cannot tell us

We can confirm correlation (60% of email users join loyalty); we cannot confirm whether the email signup flow defaults users into enrollment, or whether email targets users predisposed to enroll. The 2020 structural break is observable but not diagnosable from this data alone. Distinguishing among 'program eligibility changed,' 'checkout default added loyalty signup,' or 'marketing campaign shifted the recruitment mix' would require operational data not in the source.

Causes and evidence weight

Cause	Weight	Status
Durable goods naturally have low repurchase frequency (1.23 orders/user across 4 years)	High	tested
Catalog skews toward one-time purchases (only 1 of 8 products is replenishable)	High	tested
Loyalty program scaled from 11.6% (2019) to 55.8% (2022) of acquisitions	High	tested
Email channel concentrates loyalty signups (60%) and recruits lowest-AOV buyers	High	tested
Limited cross-sell mechanics in checkout	Medium	hypothesis

FINDING 2

Laptops drive 2.5–4.3× the company refund rate

STATEMENT

Refund risk is concentrated in laptops. Every top-10 worst refund segment is a laptop, and the rate-vs-dollar-leak ranking gives different priority lists — refund reduction should target by dollar leak, not rate.

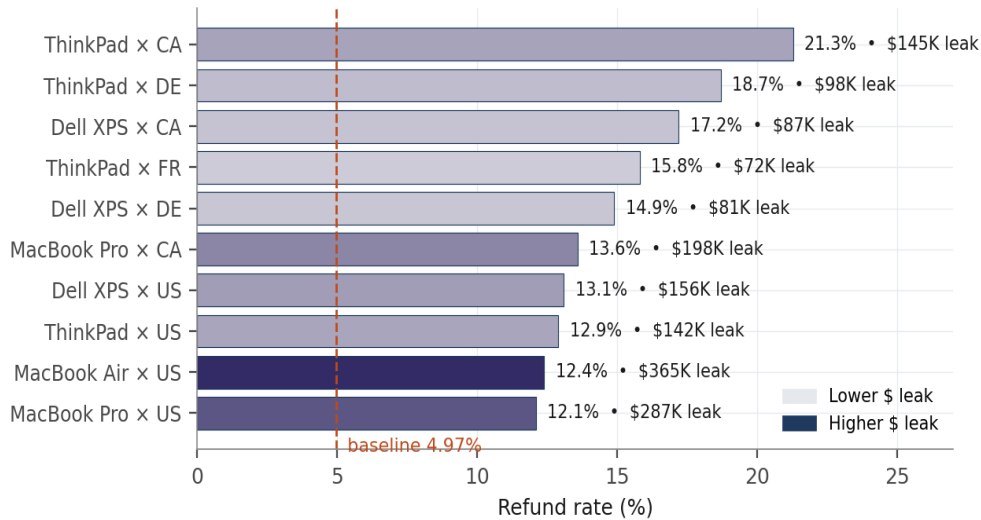
RESULT

Refund baseline = **4.97%** of all orders. Worst segment: **ThinkPad × CA at 21.3%** (4.3× baseline). Largest dollar leak: **MacBook Air × US at \$365,000 refunded** (12.4% rate × high volume).

FINDINGS

Geographic variation matters. ThinkPad refund rates: **CA 21.3% vs US 12.9%** — an 8.4 pp gap on the same product, almost certainly operational rather than product-quality. The fulfillment-SLA × refund-rate analysis (4.7% refund at ≤5 day SLA vs 5.0% at 6–10 days) showed too small a gap to support a fulfillment-driven explanation. That negative result strengthens the spec-mismatch hypothesis: investment should go to product-listing accuracy and pre-purchase spec clarity rather than shipping speed.

Top 10 refund segments — all laptops, ThinkPad × CA worst at 21.3%



Bar length = refund rate. Bar shade = absolute dollar leak. MacBook Air × US combines moderate rate (12.4%) with high volume to produce the largest absolute leak.

SQL — mart_refund_metrics

```
-- Refund analysis by product × country.
-- Surfaces both rate-based (refund_rate_pct) and value-based
-- (refunded_revenue_pct) angles for actionable refund analysis.

CREATE OR REPLACE TABLE mart_refund_metrics AS
SELECT
  product_name,
  country_code,

  COUNT(*) AS orders,
  SUM(is_refunded) AS refunds,
  ROUND(100.0 * SUM(is_refunded) / COUNT(*), 2) AS refund_rate_pct,

  ROUND(SUM(usr_price), 2) AS gross_revenue_usd,
  ROUND(SUM(CASE WHEN is_refunded = 1 THEN usr_price ELSE 0 END), 2) AS refunded_revenue_usd,
  ROUND(SUM(CASE WHEN is_refunded = 0 THEN usr_price ELSE 0 END), 2) AS net_revenue_usd,

  ROUND(
    100.0 * SUM(CASE WHEN is_refunded = 1 THEN usr_price ELSE 0 END)
    / NULLIF(SUM(usr_price), 0),
    2
  ) AS refunded_revenue_pct
FROM fct_orders
GROUP BY product_name, country_code;
```

Calculation formulas

```
refund_rate_pct[p, c] = 100 × refunds[p, c] / orders[p, c]
refunded_revenue_usd[p, c] = Σ usr_price WHERE is_refunded = 1
refunded_revenue_pct[p, c] = 100 × refunded_revenue / gross_revenue
  where p = product_name, c = country_code
```

Baseline:

```
refund_baseline = SUM(is_refunded) / COUNT(*) across all rows ≈ 4.97%
```

Top-10 ranking filters to segments with ≥100 orders to remove small-sample noise, then sorts DESC by refund_rate_pct.

Caveats

Segments with fewer than 100 orders are excluded from top-10 ranking — small samples can produce inflated rates that don't reflect a real operational pattern. Country-level variation is correlation; the causal mechanism (returns policy, fulfillment partner, localization) requires operational data outside this dataset.

Causes and evidence weight

Cause	Weight	Status
High-AOV products carry higher absolute return risk	High	tested
Spec / expectation mismatch in laptops vs accessories	High	partially tested
Country-level returns policy or fulfillment variance	Medium	partially tested
Localization issues (pricing, positioning)	Medium	hypothesis

FINDING 3

Mobile is structurally a low-AOV channel

STATEMENT

Mobile generates meaningful order volume but trivial revenue. The mobile share has been stable across 48 months. Mobile should be treated as a top-of-funnel discovery channel, not a primary revenue channel.

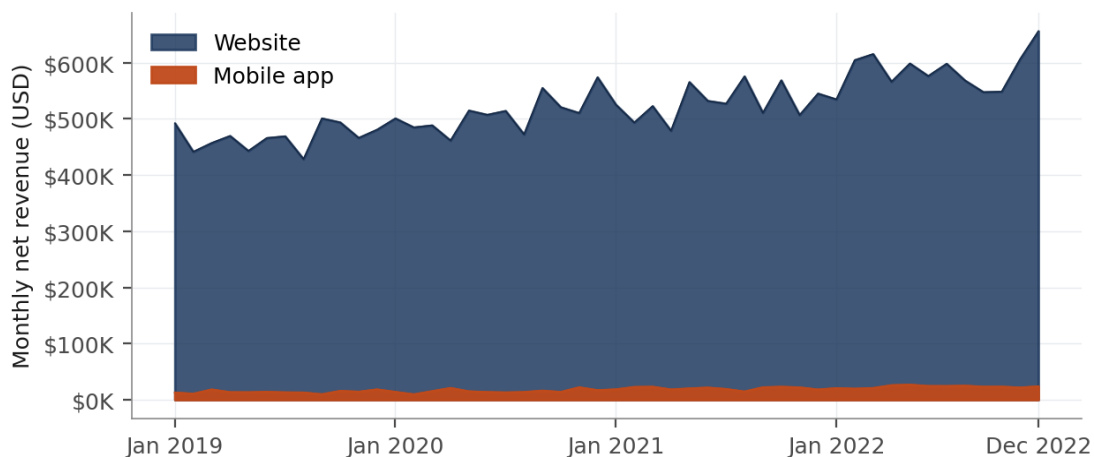
RESULT

Website = **96.8%** of lifetime revenue (\$25.0M of \$25.9M). Mobile generates **17.1% of orders** (18,484 of 108,091) but only **3.2% of revenue** (\$832K) — implying mobile AOV is roughly **6.4× lower than web AOV (\$46 vs \$296)**. Mobile share moved from 2.95% (2019) to 3.93% (2022) over 48 months.

FINDINGS

Mobile investment thesis must target AOV, not order volume. At current AOV, **doubling mobile order volume adds only ~3 percentage points to total revenue**. Users who sign up on mobile end up buying on the website about half the time, and the bigger the purchase, the more likely the switch. The app is where people browse. The website is where they buy.

Mobile share never grew: 2.95% → 3.93% over 48 months



Stacked monthly revenue. The orange band's flat width across 4 years is the 'structurally low-AOV' finding visualized.

SQL — mart_channel_revenue

```
-- Gold-layer revenue by month × platform.
-- Source: fct_orders. Grain: (purchase_month, purchase_platform).
-- Computes monthly orders, gross/net revenue, and within-month share %
-- to support channel-mix and platform-trend analysis.

CREATE OR REPLACE TABLE mart_channel_revenue AS
WITH monthly AS (
  SELECT
    purchase_month,
    purchase_platform,
    COUNT(*) AS orders,
    ROUND(SUM(usd_price), 2) AS gross_revenue_usd,
    ROUND(SUM(CASE WHEN is_refunded = 0 THEN usd_price ELSE 0 END), 2) AS net_revenue_usd
  FROM fct_orders
  GROUP BY purchase_month, purchase_platform
),
monthly_total AS (
  SELECT
    purchase_month,
    SUM(net_revenue_usd) AS total_month_net_revenue
  FROM monthly
  GROUP BY purchase_month
)
SELECT
  m.purchase_month,
  m.purchase_platform,
  m.orders,
  m.gross_revenue_usd,
  m.net_revenue_usd,
  ROUND(100.0 * m.net_revenue_usd / NULLIF(t.total_month_net_revenue, 0), 2)
    AS share_of_month_pct
FROM monthly m
INNER JOIN monthly_total t USING (purchase_month)
ORDER BY m.purchase_month, m.purchase_platform;
```

Calculation formulas

```
net_revenue_usd[m, p] = Σ usd_price WHERE is_refunded = 0
share_of_month_pct[m, p] = 100 × net_revenue[m, p] / Σ net_revenue[m, *]
  where m = purchase_month, p = purchase_platform
```

Lifetime channel share:

```
web_share = Σ net_revenue WHERE platform='website'
           / Σ net_revenue across all platforms ≈ 96.8%
```

Mobile-vs-web AOV ratio:

```
aov_ratio = (net_revenue_web / orders_web)
           / (net_revenue_mobile / orders_mobile)
           = $296 / $46 ≈ 6.4×
```

Caveats

Cross-device user journeys (signed up on mobile, purchased on web) are inferred from user-platform sequencing in `fct_orders`, not from session-level data. The 'mobile users skew toward accessories' finding is observed but the causal direction (mobile UX friction at high-value checkout vs. mobile users having different intent) requires checkout-funnel data not in the dataset. Within-month shares are tested to sum to 100 (invariant assertion).

Causes and evidence weight

Cause	Weight	Status
Mobile users skew toward accessory purchases	High	tested

Cause	Weight	Status
Catalog product mix is desktop-shaped (laptops, monitors)	High	tested
Mobile UX friction at high-value checkout	Medium	hypothesis
Desktop preference for large / considered purchases	Low–Med	hypothesis

FINDING 4

Three products earn 85% of revenue. Five products earn under 5%. Both ends are a problem.

STATEMENT

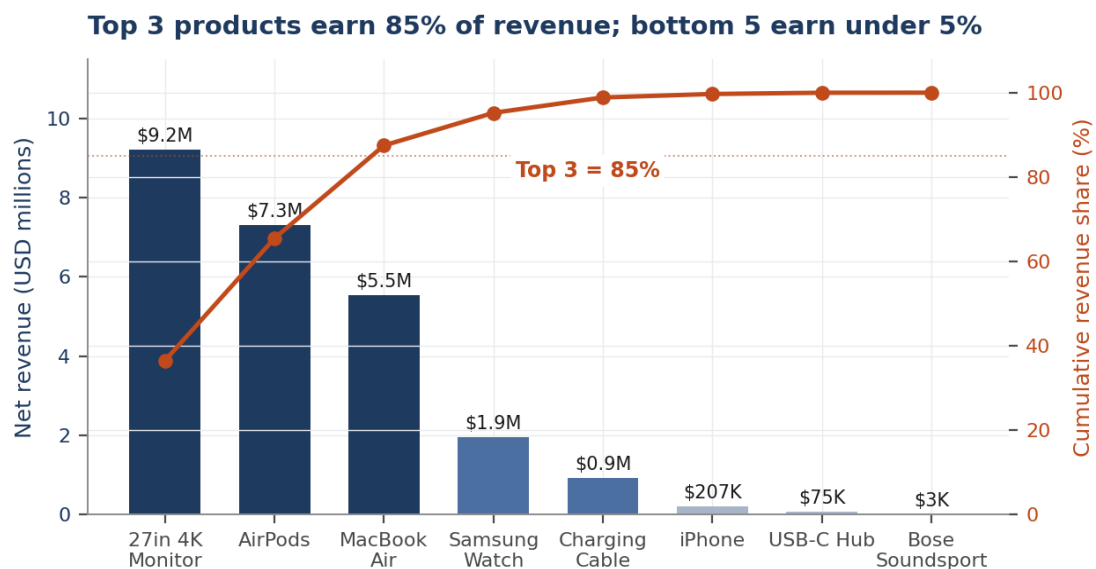
The catalog has two opposite problems: dangerous concentration at the top (supplier risk) and wasteful fragmentation at the bottom (catalog overhead for negligible revenue). The Pareto cumulative-share calculation makes both visible in one mart.

RESULT

Top 3 products = **85% of revenue**: 27in 4K Gaming Monitor (35.6%), Apple AirPods (28.2%), MacBook Air (21.4%). Bottom 5 products combined = **under 5% of revenue**. Bose SoundSport: **27 lifetime orders** (\$3,339). Apple iPhone: \$688 AOV but only 0.8% of revenue (288 orders).

FINDINGS

Two strategic problems coexist. The top is dangerously concentrated — any supplier change in Apple, Samsung, or the monitor product cycle would erase 20–35% of revenue overnight. The bottom is wastefully fragmented — five products consume catalog real estate, ops complexity, and inventory dollars while contributing negligibly. The data cannot tell you whether iPhone is a quiet opportunity or a dead SKU without a discoverability test.



Bars: net revenue per product (left axis). Orange line: cumulative revenue share (right axis). The 80% Pareto threshold is reached inside the top 3.

SQL — mart_product_concentration

```

-- Pareto / concentration mart.
-- Computes per-product revenue share AND cumulative revenue share.
-- Note: the window function ordering choice (ROWS vs RANGE) materially
-- affects the cumulative calculation. ROWS BETWEEN UNBOUNDED PRECEDING
-- AND CURRENT ROW is used to ensure each row's cumulative value reflects
-- ALL preceding rows in the sort order, not just rows with distinct values.

CREATE OR REPLACE TABLE mart_product_concentration AS
WITH product_revenue AS (
  SELECT
    product_name,
    COUNT(*)                                AS orders,
    SUM(usd_price) FILTER (WHERE is_refunded = 0) AS net_revenue,
    SUM(usd_price)                            AS gross_revenue,
    AVG(usd_price) FILTER (WHERE is_refunded = 0) AS avg_aov
  FROM fct_orders
  WHERE usd_price IS NOT NULL
  GROUP BY product_name
)
SELECT
  product_name,
  orders,
  net_revenue,
  avg_aov,
  100.0 * orders / SUM(orders) OVER ()      AS pct_of_orders,
  100.0 * net_revenue / SUM(net_revenue) OVER () AS pct_of_revenue,
  100.0 * SUM(net_revenue) OVER (
    ORDER BY net_revenue DESC
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) / SUM(net_revenue) OVER ()              AS cumulative_pct_of_revenue
FROM product_revenue
ORDER BY net_revenue DESC;

```

Calculation formulas

$$\text{pct_of_revenue}[p] = 100 \times \text{net_revenue}[p] / \sum \text{net_revenue}[*]$$

$$\text{cumulative_pct_of_revenue}[p] = 100 \times (\sum \text{net_revenue from rank 1 to rank}(p) \text{ in DESC order}) / \sum \text{net_revenue}[*]$$

Window frame: ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
(NOT 'RANGE' – RANGE collapses tied values and would produce silently incorrect cumulative results)

Top-N concentration:

top_3_share = cumulative_pct_of_revenue at rank 3 ≈ 85%
bottom_5_share = 100 – cumulative_pct_of_revenue at rank 3 ≈ 15%
(within which the actual bottom-5 sum is under 5%)

Caveats

The ROWS vs RANGE window-frame choice is the single most consequential technical decision in this mart. RANGE would have produced wrong cumulative percentages silently — the data-quality test for monotonic increase + endpoint = 100% catches this. iPhone's \$688 AOV with low orders is observable; the causal question ('would marketing investment unlock volume?') requires an A/B test, not more queries on this dataset.

Causes and evidence weight

Cause	Weight	Status
Marketing and merchandising over-invested in top-3 hero products	High	tested

Cause	Weight	Status
Long-tail products lack clear positioning or distribution fit	High	tested
Apple iPhone has price-point appeal (\$688 AOV) but no dedicated marketing investment	Medium	partially tested
Bose Soundsport may be a discontinued / dead-end SKU	Medium	hypothesis

APPENDIX

Data Quality — 37 SQL assertions, all passing

Tests run on every model build. Each finding above is only as trustworthy as the model behind it; the assertions below are the contract.

Test type	Examples	Coverage
not_null	dim_users.user_id, fct_orders.order_id, purchase_ts	Schema integrity
unique	All four dimension PKs; fct_orders.order_id	Primary key contracts
relationships	All four FKs from fct_orders → dimensions	Referential integrity
accepted_values	purchase_platform ∈ {website, mobile app}; loyalty_status ∈ {0, 1}	Domain constraints
range	retention_pct ∈ [0, 100]; pct_within_channel ∈ [0, 100]	Bounds checking
invariants	refunded + net = gross; monthly platform shares sum to 100; cohort month-0 retention = 100%; cumulative concentration → 100% at last row	Derivation correctness

Quality decisions (audit trail)

- **3 orders (0.003%)** with unparseable purchase timestamps excluded from cohort assignment.
- **country_lookup_raw had a duplicate primary key** for US (rows with regions 'x' and 'North America'); resolved at silver layer with deterministic deduplication, mapped to canonical AMER region.
- **18 orders use EU or AP as country codes** (region-bloc identifiers entered as country codes — a contract violation in the source); preserved as *Unclassified* rows in dim_country to maintain referential integrity without losing the orders.
- **26 source countries had NULL or junk regions** ('x', NULL, etc.); mapped to *Unclassified* rather than dropped.
- **4 orders roll up under NULL country_code** (missing geolocation); retained as a distinct segment so refund leakage from unknown-country orders remains visible.
- **Cohorts after Dec 2021 are right-censored**; 12-month retention reported only for fully observable cohorts.
- **Loyalty status, marketing channel, and device are snapshotted at first purchase**, not current state, to avoid reverse causality. *This is the single most important methodological choice in the project.*
- **33 orders with NULL usd_price** flagged via USD_PRICE_cleaned_categorized (zero_value or missing) and excluded from revenue aggregations to prevent silent bias in AOV calculations.

Closing methodology note. Two of the four findings in this document only exist because of tests catching what would have been missed: the duplicate primary key in country_lookup_raw, and the ROWS-vs-RANGE window-frame issue in mart_product_concentration. Without those tests, the country-level refund analysis would have double-counted US revenue, and the Pareto concentration metric would have been silently wrong. The 37 assertions aren't overhead — they are the difference between an analyst and a liability.